

Understanding d-LIN

The problem, the assumption, and the encryption idea

Prajwal Saxena

1 Start with sparse parity equations

The central observation is

$$b = Mx + e \quad \text{over } \mathbb{F}_2.$$

First evaluate several short XOR equations on a hidden assignment x . Then independently flip some of their output bits. The observer sees the equations and the possibly corrupted outputs, but not the assignment or the locations of the flips. This is the noisy d -LIN setting in the supplied notes. [1, 2]

What do the symbols mean?

Symbol	Meaning
n	Number of binary variables in the hidden assignment.
m	Number of equations, also called clauses.
d	Number of distinct variables used in each equation.
$M \in \mathbb{F}_2^{m \times n}$	Public matrix describing which variables occur in each equation.
$x \in \mathbb{F}_2^n$	Hidden assignment; each coordinate is 0 or 1.
$e \in \mathbb{F}_2^m$	Error vector: $e_i = 1$ means that output i was flipped.
$b \in \mathbb{F}_2^m$	Observed output vector, equal to $Mx + e$.

Why work over $\mathbb{F}_2$?

The field $\mathbb{F}_2 = \{0, 1\}$ uses arithmetic modulo 2. Addition is XOR:

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 1 = 0.$$

Thus $x_1 + x_2 + x_4 = 1$ says that an odd number of these three bits are 1. Subtraction is also addition, so $Mx - b$ and $Mx + b$ mean the same thing.

What does “ d -sparse” mean here?

Every row of M has *exactly* d ones. For example, the row $(1, 1, 0, 1, 0)$ selects x_1, x_2, x_4 , so it represents a 3-LIN equation. Sparsity is a property of each *row*; it does not say that each variable occurs only d times.

The first distinction

Exact d -LIN equations can be solved by Gaussian elimination. The cryptographic question concerns **random sparse equations with unknown output errors**, not the difficulty of ordinary linear algebra.

2 Generate an instance and work through it

Choose the matrix, assignment, and noise independently; their distributions are part of the problem. [2]

1. For each of the m rows, choose d distinct positions uniformly from the n variable positions and put ones there. Choose rows independently.
2. Choose the hidden assignment x uniformly from $\{0, 1\}^n$.
3. Independently sample each error bit with $\Pr[e_i = 1] = \epsilon$.
4. Compute $b = Mx + e$ and reveal (M, b) , keeping x and e hidden.

Different rows may repeat. Column weights need not be equal. An arbitrary sparse matrix is not automatically a sample from this distribution.

The example from the drafts

Take $n = 5$, $m = 4$, and $d = 3$:

$$M = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}, \quad e = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

All calculations are modulo 2. Multiplication gives

$$Mx = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \quad b = Mx + e = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}.$$

Only the second output has been flipped. For the planted assignment x :

Row	Variables used	Clean output	Observed output	Satisfied?
1	$x_1 + x_2 + x_4$	1	1	Yes
2	$x_2 + x_3 + x_5$	0	1	No
3	$x_1 + x_3 + x_4$	0	0	Yes
4	$x_2 + x_4 + x_5$	1	1	Yes

The assignment used to generate the data satisfies three of the four observed equations. Its residual is $Mx - b = e$, which contains one 1. [1]

A useful warning hidden in this example

The different assignment $x' = (0, 1, 0, 0, 0)^T$ satisfies $Mx' = b$ exactly. So the *instance* is fully satisfiable even though the *planted assignment* violates one equation. As the second draft explains, finding a good fit and recovering the planted secret are different goals. This example illustrates arithmetic, not hardness. [2]

3 Understand noise, satisfaction, and distance

The noise rate is a probability, not a fixed error count

The usual range is $0 < \epsilon < 1/2$. Writing $e_i \sim \text{Ber}(\epsilon)$ means

$$\Pr[e_i = 1] = \epsilon, \quad \Pr[e_i = 0] = 1 - \epsilon.$$

The bits are independent. If $\text{wt}(e)$ denotes the number of ones in e , then

$$\mathbb{E}[\text{wt}(e)] = \epsilon m.$$

For $m = 1000$ and $\epsilon = 0.01$, the *expected* number of flipped outputs is 10. A sample may have fewer or more flips: this is not a promise of at most ten errors. [1, 2]

How do we count satisfied equations?

For any candidate assignment z , the residual $Mz - b$ has a 1 exactly where the candidate disagrees with an observed equation. Therefore

$$\text{fraction satisfied by } z = 1 - \frac{\text{wt}(Mz - b)}{m}.$$

An instance is called $(1 - \rho)$ -satisfiable when *some* assignment satisfies at least that fraction of its clauses:

$$\text{wt}(Mz - b) \leq \rho m \quad \text{for some } z.$$

Here ϵ is the probability used to sample noise; ρ is a bound on the actual fraction of violated clauses. Keeping different symbols avoids confusing an average with a guarantee.

For the planted x , the satisfied fraction is exactly $1 - \text{wt}(e)/m$, and its expectation is $1 - \epsilon$. This is not necessarily the best fraction attainable by another assignment. [2]

The same idea as a distance problem

The set of all noiseless outputs is the image of M :

$$\text{Im}(M) = \{Mz : z \in \mathbb{F}_2^n\}.$$

Think of these as the valid output strings. Hamming distance counts the positions in which two strings differ. Since $b = Mx + e$,

$$\underbrace{d_H(b, Mx)}_{\text{distance to the planted output}} = \text{wt}(e) \quad , \quad \underbrace{d_H(b, \text{Im}(M))}_{\text{distance to the nearest valid output}} \leq \text{wt}(e) \quad .$$

The inequality can be strict: in our example the two distances are 1 and 0. The set $\text{Im}(M)$ is also called a binary linear code, which explains the coding-theory viewpoint. [2]

Three useful boundary cases

At $\epsilon = 0$, there is no noise and an exact solution is easy to find. At $\epsilon = 1/2$, the output b is completely random and reveals no information about x . For a known $\epsilon > 1/2$, flipping every observed output changes the noise rate to $1 - \epsilon$. Thus 0.7 is equivalent to 0.3 after this relabeling; it is not simply “more secure.” [2]

4 Specify what an algorithm must do

The equation $b = Mx + e$ describes how data is generated. It does not, by itself, specify what counts as solving the problem. The drafts discuss several related tasks. [2]

Search: recover the planted assignment

Given (M, b) , output the particular hidden x used to generate b . This is the search task in ABW's definition of **Search3LIN**. It is the main task behind the 3-LIN assumption discussed on the next page.

Approximate recovery: get close to the planted assignment

Output $\hat{x}$ that differs from x in only a small fraction of its coordinates. The version in the second draft allows at most $0.1n$ incorrect secret bits. Notice that this measures errors in the *secret*, not the number of unsatisfied equations.

Good fitting: satisfy many observed equations

Find z for which $\text{wt}(Mz - b)$ is small. An optimization version asks for the smallest possible value. This task talks about agreement with the observations; without a further argument, it does not promise recovery of the planted secret.

Decision: distinguish structure from random outputs

Decide which of these two ways of generating the observation was used:

$$\underbrace{(M, Mx + e)}_{\text{noisy planted outputs}} \quad \text{or} \quad \underbrace{(M, u)}_{\text{independent uniform outputs}}.$$

The matrix has the same distribution in both cases. The decision question is whether an efficient algorithm can tell the two cases apart with the required advantage. It is not merely a test of whether $Mx = b$ has an exact solution.

ABW also studies *prediction*: using the observed equations to predict another parity associated with the secret. The paper connects specified tasks through reductions, under stated parameter and distribution conditions. They should not be treated as identical definitions. [2]

Why does Gaussian elimination not settle the noisy task?

Without noise, elimination finds a solution to $Mx = b$, or detects inconsistency. With noise, solving all the observed equations may be impossible; even when it is possible, the answer need not be the planted x , as our example shows. The unknown errors are part of the inference problem.

This explains the obstacle, not a proof of hardness. There are 2^n assignments, but a large search space does not rule out a clever algorithm: even noiseless linear systems have that same search space.

Missing information is different from computational hardness

If two secrets always give the same clean output, the observations cannot distinguish them. For example, with even d , complementing every secret bit leaves all row parities unchanged. A general recovery problem must account for such symmetries. ABW's central 3-LIN search assumption concerns its specified random ensemble and parameters. [2]

5 State the hardness assumption carefully

The underlying assumption, informally

For the specified random sparse matrix, secret, and noise distributions, and for a specified parameter regime, no efficient randomized algorithm can reliably recover the planted assignment from $(M, Mx + e)$.

This is an **average-case assumption**: success is measured over randomly generated instances and the algorithm's randomness. It does not claim that every sparse noisy system is hard. [2]

The main 3-LIN regime in ABW09

ABW09 refers here to Applebaum, Barak, and Wigderson's 7 November 2009 manuscript, *Public-Key Cryptography from Different Assumptions*. Its Assumption 5.2 concerns

$$d = 3, \quad m = C_0 n^{1.4}, \quad \epsilon = C_1 n^{-0.2}.$$

Here $C_0, C_1 > 0$ are fixed constants. In the paper's definition, intractability means that every probabilistic polynomial-time algorithm has recovery probability below $2/3$ for all sufficiently large n . Assumption 5.2 states this for every positive choice of the constants; its applications need only suitable fixed constants. [3]

An *efficient* algorithm here runs in time bounded by a polynomial in the problem size. These formulas describe growing instances, not a concrete security recommendation.

How should we read these parameters?

Every equation still uses just three variables. The number of equations grows faster than n , while the fraction of flipped outputs decreases as n grows. Nevertheless, substituting into the expected-error formula gives

$$\mathbb{E}[\text{wt}(e)] = m\epsilon = C_0 C_1 n^{1.2}.$$

The expected *number* of errors grows even though their *fraction* shrinks. A vanishing noise rate does not make these instances noiseless. [2]

What is assumed, and what is proved?

The assumption is search hardness. *Conditional on it*, the paper proves the existence of semantically secure public-key encryption: informally, efficient observers cannot learn the protected message information from the ciphertext. The construction matters; an arbitrary scheme using $Mx + e$ does not inherit this theorem. [2]

The paper analyzes restricted algorithm families, including local tests, low-degree approaches, restricted circuits, and semidefinite-programming methods in specified regimes. This evidence does not exclude every efficient algorithm. Nor does worst-case hardness of a related problem establish this average-case assumption. [2]

Is d-LIN the only assumption used in every ABW route?

No. The 3-LIN-only route uses the regime above. A separate route also assumes *Decisional Unbalanced Expansion* (DUE), concerning the difficulty of detecting a specially planted graph structure. Its conditions are different. [2]

Primary-paper anchors: Definition 5.1 and Assumption 5.2, p. 12; Theorems 2–3, p. 5; Corollary 5.6, p. 15. These are printed pages of the 57-page manuscript.

6 See how a short dependency enables decryption

Key generation produces a sparse matrix together with a hidden dependency among a small number of its rows. This requires more than choosing an arbitrary sparse matrix. [2]

Public key and secret key

The public key is a matrix M . The secret key includes a small set of row indices S such that

$$\sum_{i \in S} M_i = 0, \quad m \in S.$$

The last row belongs to S . XORing the rows in S cancels every variable. The set S is the *trapdoor*: useful information retained during key generation.

Encrypt one bit

To encrypt a message bit $\sigma \in \{0, 1\}$, sample fresh random x and e . Let $u_m = (0, \dots, 0, 1)^\top$ and output

$$c = Mx + e + \sigma u_m.$$

For message 0, send the noisy output unchanged. For message 1, flip its final bit. Here x is *fresh encryption randomness*, not the long-term secret key. [2]

Decrypt by XORing the secret coordinates

The receiver computes

$$\begin{aligned} \hat{\sigma} &= \sum_{i \in S} c_i \\ &= \left(\sum_{i \in S} M_i \right) x + \sum_{i \in S} e_i + \sigma \\ &= \sigma + \sum_{i \in S} e_i. \end{aligned}$$

The hidden assignment disappears. The message appears once because S contains the last row. These are the encryption and decryption identities in ABW's basic bit scheme. [3]

Why must the dependency be short?

Decryption is correct whenever the noise bits on S have even parity. In particular, it is correct if none of them is 1. For $r = |S|$,

$$\Pr[\text{decryption error}] \leq r\epsilon.$$

Thus a short dependency and a small enough noise rate give a useful error bound. Noise elsewhere in the ciphertext does not enter this particular decryption sum.

Gaussian elimination can find ordinary row dependencies. The point is not to hide the existence of *all* linear relations, but to retain a useful *short* relation. Long relations combine many noise bits and can lose the signal needed to decode. [2]

The identity is not the whole security proof

Security requires an argument about the public-key distribution and possible attacks. ABW also transforms its weak bit scheme into semantically secure encryption. Cancellation alone does not establish message privacy. [2]

7 Connect the ideas and keep the distinctions

Where do LPN, LWE, and Module-LWE fit?

These problems share a noisy-linear pattern, but not the same hardness assumption. [1, 2]

Problem	Main features in the usual formulation
d -LIN	Binary arithmetic; exactly d ones in each random row; independent output flips.
LPN	Binary arithmetic and output flips, but standard LPN uses uniformly random binary rows rather than fixed small row weight.
LWE	Linear equations modulo an integer q , with a specified small modular error distribution.
Module-LWE	A structured version using vectors and matrices of polynomial-ring elements, together with a specified error distribution.

LPN is the closest comparison: both use corrupted binary parities. But replacing dense random rows by sparse ones changes the distribution, so hardness does not transfer automatically.

For LWE, “small error” concerns its magnitude modulo q , not a small fraction of flipped bits. Module-LWE adds algebraic structure. Each setting needs its own security argument. [2]

Can we change the field?

Sparse noisy linear equations can be defined over other finite fields, but the noise model and hardness claim must be specified again. The appeal of $\mathbb{F}_2$ here is that equations are XOR constraints, errors are output flips, and adding a bit to itself cancels it. Changing the field is not automatically a security improvement. [1, 2]

Four quick checks

Does $\mathbb{E}[\text{wt}(e)] = 10$ guarantee at most ten errors? No; it is an average.

Does fitting all observed equations recover the planted x ? Not necessarily; the worked example gives a different exact-fitting assignment.

Does having 2^n possible secrets prove hardness? No; noiseless systems also have that many candidates before the equations are used.

Does the decryption identity prove security? No; the sampling distribution, parameter regime, and security reduction are also essential. [2]

The central picture

A d -LIN instance is a collection of short parity equations whose outputs may have been flipped. The assumption is about recovering the planted assignment from *random instances in a specified regime*. The encryption construction adds a short secret dependency that lets the legitimate receiver cancel the random assignment.

Sources

- [1] *Understanding the d -LIN Problem: Sparse Noisy Linear Equations over $\mathbb{F}_2$* . Supplied draft, **Pasted text.txt**. Basis for the introductory notation and worked example.
- [2] *Understanding d -LIN through ABW09*. Supplied revised draft, **Pasted text (2).txt**. Basis for the distinctions about noise, recovery, assumptions, and encryption.
- [3] B. Applebaum, B. Barak, and A. Wigderson. *Public-Key Cryptography from Different Assumptions*, 7 November 2009. The main assumption and basic encryption identities were checked against this manuscript; other exposition is adapted from the supplied drafts.